

Question 1

Not yet
answeredMarked out of
4.00

Flag question

Given a linear model with explicit bias term $g(\mathbf{x}; \mathbf{w}, b)$, we want to obtain an update rule for b similar to the obtained update rule for $\mathbf{w}$: $\mathbf{w}^{new} = \mathbf{w}^{old} - \eta \frac{\partial}{\partial \mathbf{w}} R_{emp}(\mathbf{y}, \mathbf{x}, \mathbf{w}, b)$.

Note that $R_{emp}(\mathbf{y}, \mathbf{x}, \mathbf{w}, b) = \frac{1}{2} \sum_{n=1}^N (y^n - (\mathbf{w}^T \mathbf{x}^n + b))^2$

Select one:

- ☐ a. $b^{new} = b^{old} - \eta \frac{1}{2} \sum_{n=1}^N ((\mathbf{w}^T \mathbf{x}^n + b) - y^n) \mathbf{x}^n$
- ☐ b. $b^{new} = b^{old} - \eta \frac{1}{2} \sum_{n=1}^N ((\mathbf{w}^T \mathbf{x}^n + b) - y^n)$
- ☐ c. $b^{new} = b^{old} - \eta \sum_{n=1}^N ((\mathbf{w}^T \mathbf{x}^n + b) - y^n) \mathbf{x}^n$
- ☒ d. $b^{new} = b^{old} - \eta \sum_{n=1}^N ((\mathbf{w}^T \mathbf{x}^n + b) - y^n)$

Question 2

Not yet
answeredMarked out of
3.00

Flag question

As described in the lecture, for single-layer neural networks with quadratic loss, we have the following objective function $R(b) = \frac{1}{2} \sum_{n=1}^N (y^n - (\mathbf{w}^T \mathbf{x}^n + b))^2$, where we made the bias parameter b explicit. Which bias parameter optimizes the objective function assuming fixed $\mathbf{w}$?

Select one:

- ☒ a. $b = \sum_{n=1}^N (y^n - (\mathbf{w}^T \mathbf{x}^n))$
- ☒ b. $b = \frac{1}{N} \sum_{n=1}^N (y^n - (\mathbf{w}^T \mathbf{x}^n))$ b)
- ☐ c. $b = \frac{1}{2N} \sum_{n=1}^N (y^n - (\mathbf{w}^T \mathbf{x}^n))$
- ☐ d. $b = \frac{2}{N} \sum_{n=1}^N (y^n - (\mathbf{w}^T \mathbf{x}^n))$
- ☐ e. $b = \frac{1}{N} \sum_{n=1}^N (x^n - (\mathbf{w}^T \mathbf{y}^n))$

2.

Question 3

Not yet
answered

Marked out of
3.00

Flag question

What is the objective function of the perceptron?

Select one:

- ☐ a. $-\sum_{n=1, y^n=1}^N y^n \mathbf{w}^T \mathbf{x}^n$
- ☐ b. $-\sum_{n=1, y^n=-1}^N y^n \mathbf{w}^T \mathbf{x}^n$
- ☒ c. $-\sum_{n=1, a^n y^n=-1}^N y^n \mathbf{w}^T \mathbf{x}^n$ where $a^n = \text{sign}(\mathbf{w}^T \mathbf{x}^n)$
- ☐ d. $-\sum_{n=1}^N y^n \mathbf{w}^T \mathbf{x}^n$
- ☐ e. $-\sum_{n=1, a^n y^n=1}^N y^n \mathbf{w}^T \mathbf{x}^n$ where $a^n = \text{sign}(\mathbf{w}^T \mathbf{x}^n)$

Question 4

Not yet
answered

Marked out of
3.00

🚩 Flag question

Under which assumption does the optimization procedure for the Perceptron converge?

Select one or more:

- ☐ a. It converges for all problems as long as the data is normalized
- ☐ b. It is never guaranteed to converge
- ☐ c. It converges only if the data is in range $[0, 1]$ and linearly separable
- ☒ d. It converges for linearly separable problems after finite many steps
- ☐ e. It converges for all problems after finite many steps

Question 5

Not yet
answeredMarked out of
4.00

Flag question

Write down the recursion formula for delta-errors of the backpropagation algorithm using the notation of the lecture.

$$\delta^{l-1} = \dots$$

Select one:

☐ a. $\delta^{l-1} = \delta^l \mathbf{W}^{l-1} \text{diag}(f'(s^{l-1})) f(s^{l-1})$

☐ b. $\delta^{l-1} = \delta^l \mathbf{W}^{l-1} \text{diag}(f'(s^{l-1})) s'^{l-1}$

☒ c. $\delta^{l-1} = \delta^l \mathbf{W}^l \text{diag}(f'(s^{l-1}))$

☐ d. $\delta^{l-1} = \delta^l \mathbf{W}^l \text{diag}(f'(s^l))$

☐ e. $\delta^{l-1} = \delta^{l-1} \mathbf{W}^{l-1} \text{diag}(f'(s^{l-1}))$

Question 6

Not yet
answered

Marked out of
2.00

🚩 Flag question

Mark all activation functions that map all inputs to strictly **positive** values!

Select one or more:

- ☐ a. Linear
- ☐ b. SELU
- ☒ c. ReLU
- ☒ d. Sigmoid nur d)
- ☐ e. Tanh

Question 7

Not yet
answered

Marked out of
2.00

🚩 Flag question

Mark all activation functions whose derivative is not continuous!

Select one or more:

- ☐ a. Tanh
- ☒ b. SeLU
- ☐ c. Sigmoid
- ☐ d. Linear
- ☒ e. ReLU

Question 8

Not yet
answered

Marked out of
2.00

🚩 Flag question

Mark all activation functions that are strictly monotonically increasing!

Select one or more:

- ☒ a. Linear
- ☒ b. Sigmoid
- ☐ c. ReLU
- ☒ d. Tanh
- ☒ e. SeLU

Question 9

Not yet
answered

Marked out of
2.00

🚩 Flag question

Mark all activation functions that are bounded from above and below!

Select one or more:

- ☒ a. Tanh
- ☒ b. Sigmoid
- ☐ c. Linear
- ☐ d. SELU
- ☐ e. ReLU

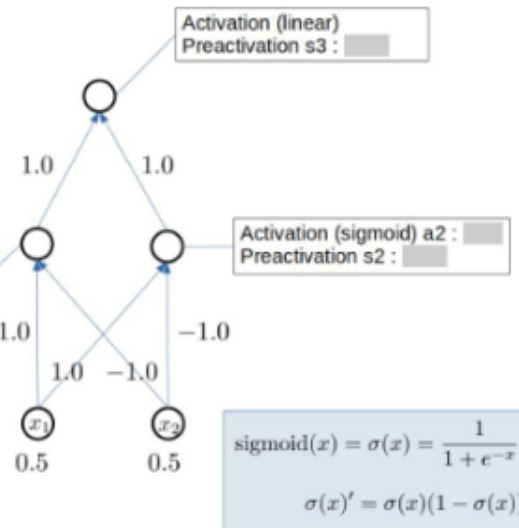
Forward pass:

prediction: $\hat{y} =$

output layer:

hidden layer:

input layer:

inputs:
 $x = (0.5, 0.5)$ 

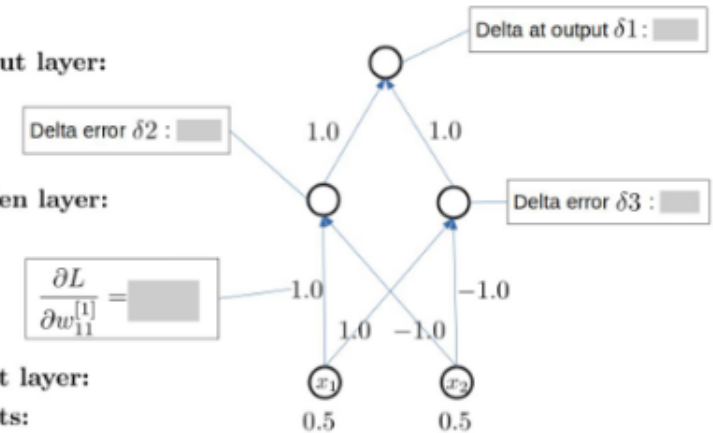
Backward pass:

loss function: $L(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$ actual labels: $y = 0$ prediction: $\hat{y} =$ (copy from forward pass)

output layer:

hidden layer:

input layer:

inputs:
 $x = (0.5, 0.5)$ 

The preactivations of the hidden layer are denoted as s_1 , and s_2 from left to right. The respective activations, are denoted as $a_1 = \sigma(s_1)$ and identical for s_2 . The preactivation of the output layer is denoted as s_3 and the output as $\hat{y}$. The delta at the output is denoted as δ_1 , and the hidden deltas as δ_2 , and δ_3 from left to right, respectively.

The gradient of the loss w.r.t. the first weight of the first layer $\frac{\partial L}{\partial w_{1,1}^{[1]}}$ is the last to calculate.

Your answer must be the output $\hat{y}$, the delta error δ_2 as well as the gradient of the loss w.r.t. the first weight of the first layer $\frac{\partial L}{\partial w_{1,1}^{[1]}}$, therefore the answer will be (without ") :

" $\hat{y}; \delta_2; \frac{\partial L}{\partial w_{1,1}^{[1]}}$ "

To give an example, your answer would look like (without ") : "1;-0.1;0.02".

Round to **3 decimal places** if necessary.

You must **separate the answers with semicolons, use dots as commas** and **not add unnecessary zeros**.

The following answers would be **incorrect**:

Trailing zeros: "1.0;-0.10;0.020"

Commas instead of dots: "1;-0,1;0,02"

Answer:

$$s_1 = 1 \cdot 0,5 + 1 \cdot 0,5 = 0 \quad s_2 = 1 \cdot 0,5 + 1 \cdot 0,5 = 0$$

$$a_1 = \sigma(s_1) = 0,5 \quad a_2 = \sigma(s_2) = 0,5$$

$$\hat{y} = 1 \cdot 0,5 + 1 \cdot 0,5 = 1$$

$$\frac{\partial L}{\partial w_{1,1}} = \delta_2 \cdot w_{1,1} = 0,25 \cdot 0,5 = 0,125 \quad \delta_3 = \hat{y} - y = 1 - 0 = 1$$

$$\delta_2 = \delta_3 \cdot f'(s_2) \cdot 1 = 0,25$$

Question 11

Not yet
answered

Marked out of
3.00

Flag question

The generalization error which is the expected loss on future data (also called risk R_{gen}) is:

Select one:

- ☒ a. A functional, i.e. an operator which maps functions to scalars
- ☐ b. A function that maps from $\mathbb{R}^N$ to $\mathbb{R}^N$
- ☐ c. A linear mapping

Question 12

Not yet
answered

Marked out of
3.00

Flag question

A special method to empirically estimate the generalization error is cross-validation. Mark the true statements about it.

Select one or more:

- ☐ a. Cross-validation should also be used on huge datasets, due to lower variance of the estimator as training data is overlapping ?
- ☒ b. Leave-one-out cross-validation is a special case, where $N = L$, therefore each fold contains only one element.
- ☐ c. We divide the dataset in L folds, train on one folds and validate on the other folds. X
- ☒ d. Cross-validation is used when we have only few datapoints and can not afford to keep a separate test set
- ☒ e. Cross-validation provides an almost unbiased estimate for the generalization error.
- ☐ f. The cross-validation error is the highest error encountered for all L different folds.

the average error

Question **13**

Not yet
answered

Marked out of
4.00

Flag question

Perform the 2D Convolution operation $*$ with a **zero-padding of 1** using the following image $\mathbf{x}$ and kernel $\mathbf{w}$:

$$\mathbf{x} = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{pmatrix}$$

and

$$\mathbf{w} = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}$$

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

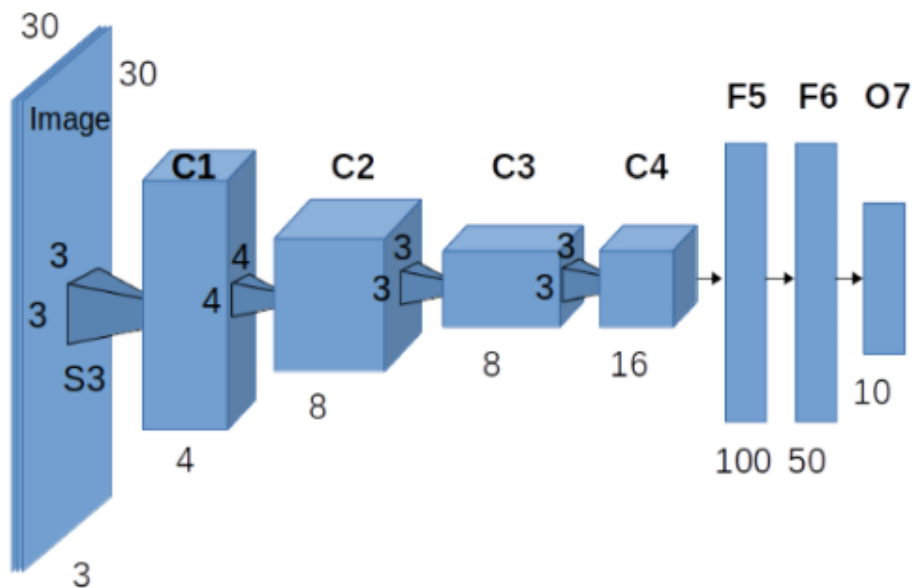
Compute $\mathbf{w} * \mathbf{x}$!

$$\begin{pmatrix} 2 & 2 & 2 \\ 2 & 4 & 3 \\ 1 & 2 & 2 \end{pmatrix}$$

Select one:

☒ e.

$$\begin{pmatrix} 2 & 2 & 2 \\ 2 & 4 & 3 \\ 1 & 2 & 2 \end{pmatrix}$$



$$P = 0$$

$$S = 1$$

$$(A - R + 2P) / S + 1$$

Layer description	Data dimension [width, height, channel] or [#features]	Number of Parameters
Input layer	[30, 30, 3]	0
C1: 3x3 convolutions with 4 kernels, stride 3	[10, 10, 4]	
C2: 4x4 convolutions with 8 kernels, stride 1	[7, 7, 8]	
C3: 3x3 convolutions with 8 kernels, stride 1	[5, 5, 8]	
C4: 3x3 convolutions with 16 kernels, stride 1	[3, 3, 16]	
F5: Fully connected layer with 100 units		
F6: Fully connected layer with 50 units		
O7: Output layer, fully connected with 10 units		

$$(8 \cdot 3 \cdot 3) \cdot 16 = 1152$$

The network above was especially created for your exam, to classify your ability to calculate the data dimensions after convolutions are applied and the number of parameters involved. It could operate on the upscaled MNIST dataset as well though. **No padding** was used in any convolutional layer. Also note that there are **no bias units** throughout the network! Images may not reflect real proportions, focus on the values rather.

Compute the dimensions after the convolutions (thus after the last convolutional layer C4) and the total number of parameters in the whole network!

The answer should be in format "[width,height,#features],#parameters" or "[#features],#parameters"!

Example answer: "[64,64,3],0" (without the "-marks")!

no +1 in
formula
for parameters

[3,3,16], 22.248

Answer:

Remark 1: Size of feature maps

- The dimensions of the feature maps are:

$$A^{[l+1]} \times B^{[l+1]} \times D^{[l+1]},$$

with $A^{[l+1]} = (A^{[l]} - R + 2P)/S + 1$ and $B^{[l+1]} = (B^{[l]} - R + 2P)/S + 1$, where

- $A^{[l]}$ is the height of the feature maps or the input image in the previous layer
- $B^{[l]}$ is the width of the feature maps or the input image in the previous layer
- P is the amount of (symmetric) zero padding,
- R is the kernel size,
- S is the stride, and
- $D^{[l+1]}$ is the number of output feature maps of the convolutional layer.

Here, the input has $l=32$ feature maps as input, $k=64$ feature maps as output, and the filter size is $n=3 \times m=3$. It is important to understand, that we don't simply have a 3×3 filter, but actually a $3 \times 3 \times 32$ filter, as our input has 32 dimensions. And we learn 64 different $3 \times 3 \times 32$ filters. Thus, the total number of weights is $n * m * k * l$. Then, there is also a bias term for each feature map, so we have a total number of parameters of $(n * m * l + 1) * k$.

- **Pooling layers:** The pooling layers e.g. do the following: "replace a 2×2 neighborhood by its maximum value". So there is no parameter you could learn in a pooling layer.
- **Fully-connected layers:** In a fully-connected layer, all input units have a separate weight to each output unit. For n inputs and m outputs, the number of weights is $n * m$. Additionally, you have a bias for each output node, so you are at $(n+1) * m$ parameters.
- **Output layer:** The output layer is a normal fully-connected layer, so $(n+1) * m$ parameters, where n is the number of inputs and m is the number of outputs.

Question 15

Not yet answered

Marked out of 3.00

 Flag question

What is the problem with Dropout concerning training-mode and test mode?

Select one:

- ☐ a. There is no problem concerning dropout concerning training and test mode.
- ☐ b. During test mode the same neurons are dropped out as during training.
- ☐ c. During testing the outputs of the backward pass need to be rescaled by the the probability that a unit activation is not dropped.
- ☒ d. During testing the outputs of the backward pass need to be rescaled by the inverse dropout probability.
- ☐ e. During testing there is a bias in the expectation of the pre-activations compared to the expectation during training with dropout.

Assume you want to develop an AI for playing a computer game. The computer game involves **green tiles** (see green parts on the image below) on a screen (white part on the image below) that have to be moved to a certain place.

You chose a convolutional neural network $g(\mathbf{x}; \mathbf{w})$ as one component of your AI to recognize whether the **green tile** is on the image $\mathbf{x} \in \mathbb{R}^{A \times B \times C}$ or not. Images have height A , width B and C channels. For this convolutional network you want to guarantee that it outputs the same prediction for all possible 90-degree-rotations of the tile and all possible positions of the tile. The usual convolution operation (stride 1, padding 1) for multiple kernels is used $\mathbf{h} = \text{Conv}(\{w_1, w_2, w_3, w_4\}, \mathbf{x})$. Each of the four kernels is convolved with the image $w_i * \mathbf{x}$, and the resulting feature maps are stacked so that $\mathbf{h} \in \mathbb{R}^{A \times B \times 4}$. We denote an element of this quantity as $h_{a,b,c}$.

In the image below, you find an example of the positive class of images. The negative class consists of images that do not contain the green tile.

Image 1								Image 2							
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	1	0	0	0	0	0	1	0	0	0	0	0
0	0	1	1	0	0	0	0	0	0	1	1	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

max-pooling?

b), c), d), e) are rotation-invariant

Pick a set of kernels (columns a, b, c, d, e, f in the image below) and neural network that **guarantee** that the neural network predicts **the same value** for **all possible locations** and **90degree-rotations** of the green tile! Note that multiple answers can be correct!

	a	b	c	d	e	f
w1	0 1 0 0 1 1 0 0 1	1 0 1 0 1 0 0 0 0	0 1 0 0 1 1 0 1 0	0 0 0 0 1 1 1 1 0	0 1 0 0 1 0 0 1 0	1 0 0 1 1 1 0 0 0
w2	1 0 0 0 1 0 1 0 1	0 0 1 0 1 0 0 0 1	0 0 1 1 1 1 0 0 0	0 1 0 0 1 1 0 0 1	0 0 0 1 1 1 0 0 0	0 1 1 1 1 0 0 0 0
w3	0 1 0 0 1 0 1 0 1	0 0 0 0 1 0 1 0 1	1 0 0 0 1 0 0 1 0	0 1 1 1 1 0 0 0 0	0 0 1 0 1 0 1 0 0	0 1 1 0 1 0 1 0 0
w4	0 0 1 1 1 1 0 0 0	1 0 0 0 1 0 1 0 0	1 1 0 0 1 0 0 0 1	1 0 0 1 1 0 0 1 0	1 0 0 0 1 0 0 0 1	0 0 1 1 1 1 0 0 0

a)	b)	c)	d)	e)	f)
$w_{1,1} = 2$	$w_{1,1} = 2$	$w_{1,1} = 3$	$w_{1,1} = 4$	$w_{1,1} = 2$	$w_{1,1} = 2$
$w_{2,1} = 2$	$w_{2,1} = 2$	$w_{2,1} = 3$	$w_{2,1} = 2$	$w_{2,1} = 2$	$w_{2,1} = 4$
$w_{3,1} = 2$	$w_{3,1} = 2$	$w_{3,1} = 2$	$w_{3,1} = 4$	$w_{3,1} = 2$	$w_{3,1} = 3$
$w_{4,1} = 3$	$w_{4,1} = 2$	$w_{4,1} = 2$	$w_{4,1} = 2$	$w_{4,1} = 1$	$w_{4,1} = 3$
$\Sigma 9$	$\text{max}=2$ $\Sigma 8$	$\Sigma 10$	$\Sigma 12$	$\text{max}=2$ $\Sigma 7$	$\text{max}=4$ $\Sigma 12$
$w_{1,2} = 4$	$w_{1,2} = 2$	$w_{1,2} = 3$	$w_{1,2} = 2$	$w_{1,2} = 2$	$w_{1,2} = 3$
$w_{2,2} = 2$	$w_{2,2} = 2$	$w_{2,2} = 2$	$w_{2,2} = 4$	$w_{2,2} = 2$	$w_{2,2} = 2$
$w_{3,2} = 3$	$w_{3,2} = 2$	$w_{3,2} = 2$	$w_{3,2} = 2$	$w_{3,2} = 1$	$w_{3,2} = 2$
$w_{4,2} = 2$	$w_{4,2} = 2$	$w_{4,2} = 3$	$w_{4,2} = 4$	$w_{4,2} = 2$	$w_{4,2} = 2$
$\Sigma 11$	$\text{max}=2$ $\Sigma 8$	$\Sigma 10$	$\Sigma 12$	$\text{max}=2$ $\Sigma 7$	$\text{max}=3$ $\Sigma 9$
$\neq$	$=$	$=$	$=$	$=$	$\neq$

Select one or more:

- ☐ 1. Kernel set: **a** and neural network $g: \hat{y} = \sigma(\frac{1}{AB} \sum_{a=0}^{A-1} \sum_{b=0}^{B-1} (\max_c h_{a,b,c}))$, where $\max_c$ indicates that the maximal value of all featuremaps/channels is chosen. **X**
- ☒ 2. Kernel set: **b** and neural network $g: \hat{y} = \sigma(\max_{a,b}(\max_c h_{a,b,c}))$, where $\max_c$ indicates that the maximal value of all featuremaps/channels is chosen and $\max_{a,b}$ indicates the maximal value of all positions. **? formula?**
- ☒ 3. Kernel set: **c** and neural network $g: \hat{y} = \sigma(\frac{1}{AB} \sum_{a=0}^{A-1} \sum_{b=0}^{B-1} (\max_c h_{a,b,c}))$, where $\max_c$ indicates that the maximal value of all featuremaps/channels is chosen.
- ☒ 4. Kernel set: **d** and neural network $g: \hat{y} = \sigma(\frac{1}{AB} \sum_{a=0}^{A-1} \sum_{b=0}^{B-1} (\max_c h_{a,b,c}))$, where $\max_c$ indicates that the maximal value of all featuremaps/channels is chosen.
- ☐ 5. Kernel set: **f** and neural network $g: \hat{y} = \sigma(\max_{a,b}(\max_c h_{a,b,c}))$, where $\max_c$ indicates that the maximal value of all featuremaps/channels is chosen and $\max_{a,b}$ indicates the maximal value of all positions. **X**
- ☒ 6. Kernel set: **e** and neural network $g: \hat{y} = \sigma(\max_{a,b}(\max_c h_{a,b,c}))$, where $\max_c$ indicates that the maximal value of all featuremaps/channels is chosen and $\max_{a,b}$ indicates the maximal value of all positions. **? formula?**

e) is rotation-invariant but strange formula
b) is rotation-invariant but strange formula

AdaGrad adapts the step size by remembering past gradients $\mathbf{g}^{(t)}$. The parameter wise learning rates are then scaled down for parameters that already experienced large gradients and up for parameters that experienced low gradients so far. For this end, we store the sum of squared past gradients $\mathbf{v}^{(t)} = \sum_{s=1}^t (\mathbf{g}^{(s)})^2$, where the squared is computed component wise. We use this to update the parameter vector $\mathbf{w}$ similar to the basic gradient descent algorithm. Select the correct formula that AdaGrad uses for this update.

Select one:

☐ a.

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + \eta \frac{1}{\sqrt{\mathbf{v}^{(t)} + \epsilon}} \odot \mathbf{g}^{(t)}$$

☐ b.

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + \eta \frac{1}{\sqrt{\mathbf{v}^{(t-1)} + \epsilon}} \odot \mathbf{g}^{(t)}$$

☐ c.

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \frac{1}{\sqrt{\mathbf{v}^{(t-1)} + \epsilon}} \odot \mathbf{g}^{(t)}$$

$\mathbf{v}^{(t)}$

☐ d.

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + \eta \frac{1}{\sqrt{\mathbf{v}^{(t)} + \epsilon}} \cdot \mathbf{g}^{(t)}$$

☐ e.

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \frac{1}{\sqrt{\mathbf{v}^{(t-1)} + \epsilon}} \cdot \mathbf{g}^{(t)}$$

☐ f.

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \frac{1}{\sqrt{\mathbf{v}^{(t)} + \epsilon}} \cdot \mathbf{g}^{(t)}$$

☒ g.

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \frac{1}{\sqrt{\mathbf{v}^{(t)} + \epsilon}} \odot \mathbf{g}^{(t)}$$

Formula (7.46)

wrong sign

no "."

Question 18

Not yet
answeredMarked out of
2.00

Flag question

The loss surface of neural networks (loss considered as the function of weights) is in general strictly convex, true or false?

Select one:

- ☐ a. True, because neural networks can approximate any function arbitrarily close according to the universal function approximator theorem.
- ☒ b. False, the convexity of the loss function depends on the choice of the optimizer
- ☐ c. False, only Sigmoid or Tanh activation leads to strictly convex loss surfaces
- ☐ d. True, due to the stacking of linear layers and activation functions the loss surface is strictly convex
- ☒ e. False, due to stacking of layers the loss surface increases in complexity

Question 19

Not yet
answeredMarked out of
6.00

Flag question

Suppose we are performing gradient descent on a quadratic objective: $R(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{A} \mathbf{w}$ with symmetric matrix $\mathbf{A}$. The dynamics of gradient descent with learning rate η can be analyzed in terms of the spectral decomposition $\mathbf{A} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^T$, where $\mathbf{Q}$ is an orthogonal matrix containing the eigenvectors of $\mathbf{A}$, and $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_D)$ is a diagonal matrix containing the eigenvalues of $\mathbf{A}$ in descending order, where $\lambda_1 = \lambda_{\max}$ is the largest eigenvalue. Then we have

$$\mathbf{w}^t = \mathbf{Q}(\mathbf{I} - \eta \mathbf{\Lambda})^t \mathbf{Q}^T \mathbf{w}^0.$$

Based on this formula, what is the value of C such that the gradient descent iterates diverge for $\eta > C$ but converge for $\eta < C$?

Select one:

- ☐ a. $C = \frac{1}{D}$
- ☐ b. $C = \lambda_{\max}$
- ☒ c. $C = \frac{2}{\lambda_{\max}}$
- ☐ d. $C = 1$
- ☒ e. $C = \frac{1}{\lambda_{\max}}$

$$|1 - \eta \lambda_i| < 1$$

Question 20

Not yet answered

Marked out of 6.00

Flag question

Assume you have a convolutional layer of 4x4 filters with a total of 16 filters and stride 2 that is applied to inputs of dimensions [32,32,3] (height, width, features) and you want to initialize the weights of this conv-layer according to **Glorot's Initialization** scheme. Further assume that you want to use a normal distribution. What are the correct parameters for this distribution?

☐ a. $W_{ij} \sim \mathcal{N}(0, \frac{2}{4 \times 4 \times 16})$ $\hat{= I}$

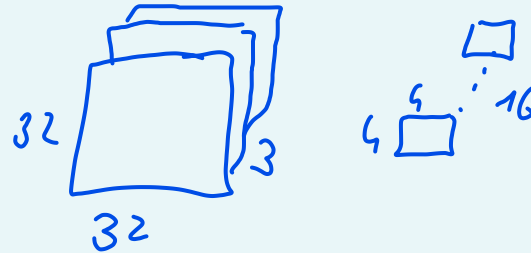
☒ b. $W_{ij} \sim \mathcal{N}(0, \frac{2}{4 \times 4 \times 3 + 4 \times 4 \times 16})$

☐ c. $W_{ij} \sim \mathcal{N}(0, \frac{1}{4 \times 4 \times 3})$ $\times$

☐ d. $W_{ij} \sim \mathcal{N}(0, \frac{2}{32 \times 32 \times 3 + 32 \times 32 \times 16})$

☐ e. $W_{ij} \sim \mathcal{N}(1, \frac{2}{4 \times 4 \times 16})$

$\mathcal{N}(0, \frac{2}{J+I})$ $\mathcal{U}(-\sqrt{\frac{6}{J+I}}, \sqrt{\frac{6}{J+I}})$



Height & width of input image don't matter

$\times$

Input J: 4.4.3

Output I: 4.4.16

Question 21

Not yet answered

Marked out of 2.00

Flag question

Inputs for a feed-forward neural network should be scaled to the interval [0, 100] - true or false?

Select one:

☐ a. True, because large numbers leads to larger gradients and therefore faster learning

☐ b. True, because large positive numbers counteract the vanishing gradient problem

☐ c. True, because gradient descent only works with positive numbers

☒ d. False, because large numbers leads to larger gradients and therefore unstable learning

☐ e. False, because gradient descent only works with negative numbers

Question 22

Not yet
answered

Marked out of
4.00

Flag question

Which of the following statements about Batch Normalization are true?

Select one or more:

- ☒ a. Each neuron learns a mean batch activation μ and standard deviation σ .
- ☒ b. Batch normalization introduces two learnable parameters β and γ for each neuron, which can scale and shift the normalized activation.
- ☐ c. In the standard formulation presented in the lecture, batch normalization aims to standardize the neuron pre-activations.
- ☐ d. Batch normalization introduces two learnable parameters β and γ for each layer, which can scale and shift the normalized activation.
- ☐ e. The network learns a single batch mean activation μ and standard deviation σ that are the same for all neurons in a layer.